Practical - 9

Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition: Implement secure user registration by inserting user details into MySQL using MySQLi. Validate input on frontend and backend, check duplicate email/username, hash passwords using password_hash(), and display appropriate messages.

A. Theory

1. What is secure user registration?

Answer: Secure registration collects user details, validates them, checks for existing accounts, and saves the new user safely in the database.

2. Why is password hashing required?

Answer: Passwords should not be stored as readable text. password_hash() creates a one-way hash; password_verify() can later verify a login attempt without needing the original password.

3. Why use prepared statements?

Answer: Prepared statements separate SQL commands from user input and reduce the risk of SQL injection.

4. Why check duplicate email or username?

Answer: It prevents multiple accounts from being created with the same unique identifier and helps keep account records consistent.

B. Implementation / Procedure

1. Database table

Create a users table with id as the primary key and email/username columns marked UNIQUE. Include name, password_hash, role, and created_at fields as needed.

2. Registration processing

Read submitted values with POST, trim text fields, validate required fields and email format, and validate password rules on the server.

3. Duplicate check

Use a MySQLi prepared SELECT statement to search for the same email or username. If found, show an error and do not insert.

4. Hash and insert

Hash the password with password_hash($password, PASSWORD_DEFAULT), then insert the user using a prepared INSERT statement.

5. Feedback

Show success only after the database insert succeeds. Display validation or database errors without exposing sensitive server details.

C. Test Cases

Test Case	Input / Action	Expected Result
Valid new user	All fields valid and email is unused	Registration succeeds and row is inserted
Invalid email	Enter malformed email	Validation error; no row inserted
Duplicate email	Register using an existing email	Duplicate warning; no duplicate row
Weak/missing password	Enter missing or invalid password	Validation error; no row inserted

D. Key Questions / Analysis / Interpretation

1. Is password_hash() used before storing passwords?

Answer: Yes. The password is hashed using PHP's password_hash() function before the value is inserted into the database. The original password is never stored in plain text.

2. Is duplicate email/username checked?

Answer: Yes. Before inserting a new account, the application checks whether the email or username already exists. If a match is found, registration is stopped and a clear message is shown.

3. Are frontend and backend validations consistent?

Answer: The frontend checks required fields and basic input format for quick feedback. The backend repeats the checks because client-side validation can be bypassed.

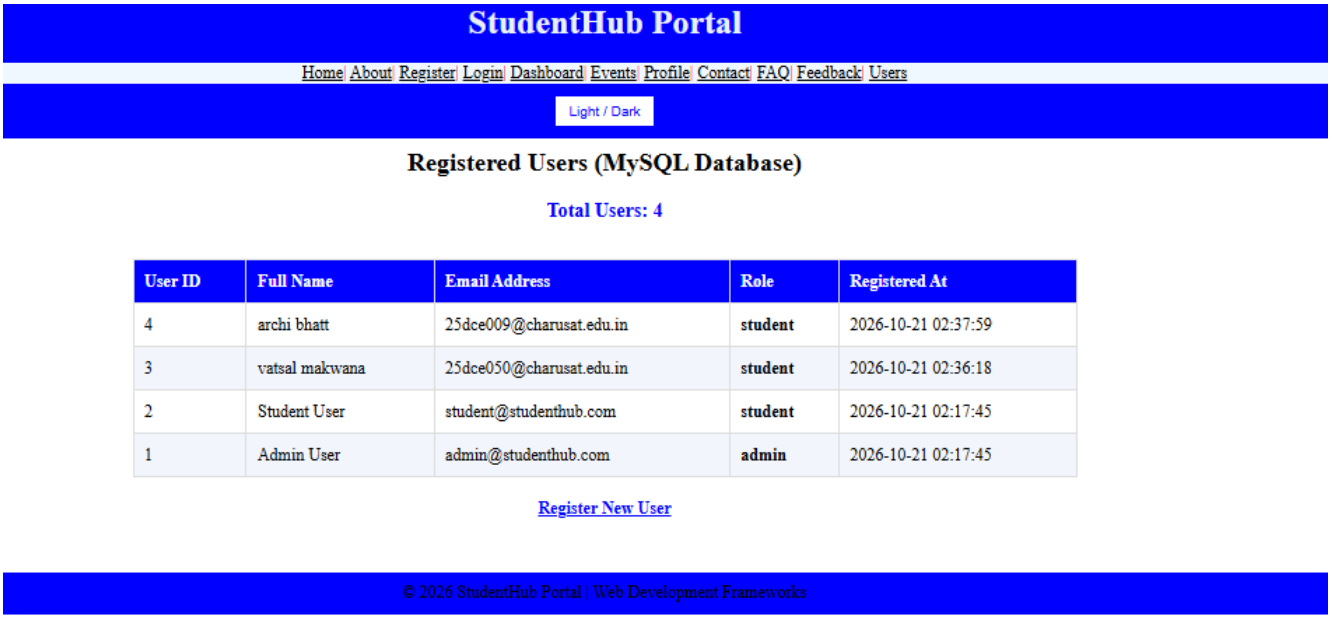
25DCE050

M. Evidence of Work (Screenshots)



N.

O.





N. GitHub Repository Link

https://github.com/vatsal-1950/StudentHUB.git